Servidores web, proxies y contenedores — versión ampliada

Para los nombres y responsabilidades esenciales, usa la versión corta. Esta guía amplía los recorridos de desarrollo y producción; describe posibilidades de despliegue, no exige montarlas todas en UD01.

Los nombres se parecen, pero no son sinónimos. La forma más segura de distinguir cada componente es preguntar qué responsabilidad asume y qué protocolo utiliza para comunicarse con el siguiente.

Responsabilidades

Componente	Responsabilidad principal	Ejemplos
Servidor web	Recibir HTTP, servir archivos y aplicar configuración web	Nginx, Apache HTTP Server, Caddy
Proxy inverso	Recibir peticiones públicas y dirigirlas a servicios internos	Nginx, HAProxy, Traefik
Balanceador	Repartir tráfico entre varias instancias	HAProxy, balanceadores cloud
Contenedor servlet	Ejecutar aplicaciones Java basadas en el contrato Servlet	Tomcat, Jetty
Servidor embebido	Vivir dentro del proceso de la aplicación	Tomcat embebido en Spring Boot
Servidor de aplicaciones	Añadir servicios empresariales estandarizados además del contenedor web	WildFly, Payara

Un producto puede asumir varias funciones. Nginx puede servir archivos y actuar como proxy. Tomcat puede recibir HTTP directamente y ejecutar servlets. La tabla clasifica responsabilidades, no limita productos.

Primero la responsabilidad, después el producto

Una petición puede atravesar varios componentes; llamarlos a todos «el servidor» impide saber dónde buscar un fallo:

- Un **servidor web** recibe una petición HTTP y puede devolver un fichero ya almacenado. Por ejemplo, Nginx puede servir una imagen sin llamar al código Java de la aplicación.
- Un **proxy inverso** recibe una petición pública y la remite a otro proceso interno. Nginx puede cumplir esta función y, al mismo tiempo, servir archivos estáticos; recibir HTTP no implica ejecutar el código de negocio.
- Un **balanceador** elige **una de varias instancias** aptas para una petición. Puede ser el mismo proxy o un componente distinto. No copia cada petición a todas las instancias.
- Un **contenedor servlet**, como Tomcat, proporciona la infraestructura web para ejecutar la aplicación Java. Los controladores Spring aportan el código de aplicación; Tomcat no es sinónimo de Spring Boot.

La palabra «contenedor» puede confundir: **Tomcat** es un contenedor web (Servlet); un **contenedor Docker** empaqueta y ejecuta procesos. Puedes poner una aplicación Spring Boot con Tomcat embebido **dentro** de Docker, pero eso no los convierte en la misma pieza. Un servidor de aplicaciones como WildFly o Payara añade otros servicios empresariales; aquí basta reconocer su papel.

Desarrollo: una aplicación accesible directamente

En el ejemplo Hello Server no hace falta instalar Nginx para empezar:

```
navegador/HTTPie --HTTP--> localhost:8080
                             |
                             v
                    Tomcat embebido + aplicación Java
                             |
                             v
                controlador -> respuesta HTML o JSON
```

`GET /` devuelve HTML creado por Java y `GET /api/hello` JSON. El navegador no ejecuta el controlador: envía la petición a Tomcat y muestra la respuesta. Que este acceso sea directo **no** significa que el proceso Java pueda prescindir de un servidor HTTP; Tomcat es ese servidor dentro de la aplicación. Es un recorrido adecuado para observar HTTP en el equipo local.

Flujo habitual con proxy inverso

```
Internet
   |
HTTPS
   v
proxy inverso
   |-- /static -> archivos
   |-- /api    -> aplicación Spring Boot :8080
   `-- /admin  -> otro servicio :8090
```

El proxy puede terminar TLS, limitar tamaños, añadir cabeceras, registrar accesos y ocultar la topología interna. La aplicación sigue siendo responsable de validar entradas y autorizar operaciones.

El mismo esquema, con dos instancias de aplicación

Una instalación de producción **podría** usar Nginx también para repartir las peticiones destinadas a `/api`:

```
cliente --HTTPS--> Nginx (termina TLS)
                     |-- /static --> archivos
                     `-- /api ----> elige una instancia
                                      |-- Spring Boot A :8080
                                      `-- Spring Boot B :8080
```

1. Nginx atiende la conexión pública; si termina TLS, descifra allí la petición HTTPS antes de enviarla al siguiente tramo.
2. Para `/static`, puede responder él mismo; para `/api`, elige una instancia disponible y le reenvía la petición. **A o B** ejecuta Java para esa petición, no las dos a la vez.
3. La instancia Java valida entradas, aplica permisos y genera el cuerpo. Nginx devuelve la respuesta al cliente.

El tramo interno puede ir por HTTP dentro de una red controlada o protegerse también con TLS si el despliegue lo exige: **terminar TLS en el proxy no cifra por sí solo toda comunicación posterior**. Un balanceador ayuda a repartir carga y tolerar fallos de una instancia, pero añade configuración, salud de instancias y registros que gestionar. UD01 pide explicar el dibujo, no montar una plataforma de producción.

No confundir comunicación entre procesos

En el ejemplo anterior, Nginx reenvía **HTTP** a una aplicación Spring Boot (`proxy_pass` en una configuración de Nginx). En la demo CGI/FastCGI, Nginx se comunica mediante **FastCGI** con `fcgiwrap` (`fastcgi_pass`). Los dos pueden estar detrás de Nginx, pero usan mecanismos distintos para llegar al código. La guía 02 muestra qué proceso queda vivo en cada caso.

Tomcat externo o embebido

Con un Tomcat externo se instala el contenedor y se despliegan aplicaciones dentro de él. Con Spring Boot, el contenedor suele empaquetarse con la aplicación y arranca desde su `main`.

Externo	Embebido
El contenedor tiene ciclo de vida propio	Aplicación y servidor forman una unidad desplegable
Puede alojar varias aplicaciones	Cada servicio suele ejecutar su instancia
Configuración central compartida	Configuración versionada con la aplicación
Despliegue tradicional mediante artefactos	Ejecución directa con `java -jar`

El modelo embebido simplifica la reproducibilidad, pero no elimina la necesidad de proxy, TLS, límites y observabilidad en producción.

Externo describe un Tomcat con ciclo de vida propio en el que se despliega una aplicación web preparada para ese contenedor. **Embebido** describe una aplicación que arranca junto con su servidor, como el `java -jar` habitual de Spring Boot. No confundas «externo» con «servidor situado en otra máquina»: la diferencia esencial es quién arranca y gestiona el contenedor web. Ambos modelos pueden estar detrás de un proxy inverso.

Señales que no demuestran la arquitectura

Cabeceras como `Server` o `X-Powered-By` pueden faltar, estar modificadas o describir solo el primer intermediario. Una CDN puede ocultar completamente la aplicación real. Sirven como indicios, no como prueba del lenguaje o mecanismo de ejecución.

Para conocer la arquitectura de un **laboratorio propio**, compara el código, la configuración y los procesos que has arrancado con lo que muestra HTTPie. Por ejemplo, una cabecera `Server: nginx` no demuestra que Nginx haya generado el JSON: podría haberlo reenviado desde un proceso Java situado detrás. Del mismo modo, `localhost:8080` es solo una dirección y un puerto, no el nombre de la tecnología que se ejecuta ahí.

Comprobación

Dibuja dos despliegues válidos para una aplicación Spring Boot:

1. Acceso directo al servidor embebido durante desarrollo.
2. Acceso de producción mediante proxy inverso y dos instancias.

Etiqueta dónde se termina TLS, dónde se ejecuta Java y dónde se balancea tráfico. Esta evidencia trabaja RA1.d.

En el dibujo de desarrollo **no hay proxy inverso** y no es necesario suponer TLS para `http://localhost:8080`. En el de producción, escribe qué ocurre con `/static` y con `/api`, por qué basta con una de las dos instancias y quién decide si el usuario tiene permiso para acceder a los datos. Si un componente deja de funcionar, indica qué otro podría seguir atendiendo y qué deberías comprobar para afirmarlo.